

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Kiểm thử Hộp trắng: Luồng dữ liệu và Độ phủ

Buổi 4 / 12 — Môn: Kiểm thử Phần mềm

Ngày 11 tháng 8 năm 2026

Nội dung buổi học

- 1 Ôn tập buổi 3: CFG và độ phủ câu lệnh / nhánh
- 2 Độ phủ điều kiện và MC/DC
- 3 Độ phức tạp cyclomatic (McCabe)
- 4 Kiểm thử luồng dữ liệu: def / use, DU-pair
- 5 Bài tập: DU-pairs trong computeScore()
- 6 Anomaly luồng dữ liệu
- 7 JaCoCo trong DigiShield: cấu hình, chạy, đọc báo cáo
- 8 Demo: nâng coverage module interception
- 9 Giới hạn của độ phủ
- 10 Thực hành, bài nộp và tài liệu tham khảo

Vị trí trong đề cương

Chương 3 — Kiểm thử hộp trắng (buổi 3–4)

- **Buổi 3:** kiểm thử luồng **điều khiển** — CFG, độ phủ câu lệnh, độ phủ nhánh, độ phủ đường đi.
- **Buổi 4 (hôm nay):** hoàn thiện Chương 3 — độ phủ **điều kiện** / MC-DC, độ phức tạp cyclomatic, kiểm thử luồng **dữ liệu**, đo độ phủ bằng **JaCoCo**.
- Buổi 5–7 (Chương 4): JUnit 5, Mockito, integration test — nơi các kỹ thuật hôm nay được *cài đặt* thành test.
- Kỹ năng hôm nay dùng trực tiếp cho BTL: báo cáo độ phủ module nhóm phụ trách.

Hệ thống thực hành

Toàn bộ ví dụ dùng code thật của DigiShield: `InterceptionServiceImpl`, `AnalyticsServiceImpl`, `ReportingServiceImpl`.

Ôn buổi 3: logic của evaluate() — can thiệp giao dịch

modules/interception/.../InterceptionServiceImpl.java

```
List<String> signals = new ArrayList<>();
if (request.onCall()) signals.add("ON_CALL");
if (request.newPayee()) signals.add("NEW_PAYEE");
boolean watchlistHit = watchRepository
    .findByTenantIdAndValue(tenantId,
        request.destAccount()).isPresent();
if (watchlistHit) signals.add("WATCHLIST_HIT");

if (request.onCall() && request.newPayee()
    && watchlistHit) {
    decision = Decision.PAUSE;    // Tam dung giao dich
} else if (watchlistHit) {
    decision = Decision.WARN;    // Canh bao nguoi dung
} else {
    decision = Decision.ALLOW;   // Cho phep
}
```

Ôn buổi 3: độ phủ nhánh chưa đủ

Buổi 3 đã đạt được

- Vẽ CFG cho evaluate()
- 100% độ phủ câu lệnh với 3 test (3 lệnh gán PAUSE/WARN/ALLOW loại trừ nhau)
- 100% độ phủ nhánh với 3 test (PAUSE / WARN / ALLOW)

Câu hỏi mở của buổi 3

Điều kiện phức onCall && newPayee && watchlistHit có 3 điều kiện con — 2 test đủ phủ cả quyết định (T/F), nhưng liệu có “nhìn thấy” từng điều kiện con?

Buổi hôm nay trả lời 3 câu hỏi

(1) Đo sâu hơn nhánh: điều kiện con, MC/DC. (2) Code phức tạp đến đâu thì cần bao nhiêu test — cyclomatic. (3) Dữ liệu chảy đúng không — data flow. Và đo tất cả bằng JaCoCo.

Decision coverage vs Condition coverage

Độ phủ quyết định (decision)

Mỗi **quyết định** (toàn bộ biểu thức boolean trong `if`) nhận giá trị **true** và **false** ít nhất một lần.

Độ phủ điều kiện (condition)

Mỗi **điều kiện con** (atomic condition) trong quyết định nhận giá trị **true** và **false** ít nhất một lần.

Ví dụ DigiShield

Quyết định: `onCall && newPayee && watchlistHit`
Ba điều kiện con: `onCall`, `newPayee`, `watchlistHit`.

Lưu ý

Condition coverage **không** bao hàm decision coverage và ngược lại — xem phần ví dụ ở slide sau.

Phản ví dụ: condition 100% nhưng decision 50% (1/2)

Ký hiệu: $A = \text{onCall}$, $B = \text{newPayee}$, $C = \text{watchlistHit}$; quyết định $D = A \wedge B \wedge C$.

Test	A	B	C	D	Kết quả
T1	T	F	T	F	không PAUSE
T2	F	T	F	F	không PAUSE

- Mỗi điều kiện con đều đã nhận cả T và F $\Rightarrow$ **condition coverage = 100%**.
- Nhưng D luôn bằng F $\Rightarrow$ nhánh PAUSE **chưa bao giờ chạy**: decision coverage = 50%.

Phản ví dụ: condition 100% nhưng decision 50% (2/2)

Ghi chú Java: short-circuit

Toán tử `&&` là *short-circuit*: với T2, B và C không được đánh giá; với T1, C cũng không (vì $A \wedge B$ đã F). Ở mức **thực thi** — mức JaCoCo đo — C chưa bao giờ được đánh giá; “100%” ở trên chỉ đúng theo bảng giá trị **lý thuyết**. JaCoCo đo ở bytecode nên “nhìn thấy” từng điều kiện con đã *được đánh giá* — và sẽ báo C chưa phủ.

Multiple condition coverage — bảng chân trị 2^n

Phủ **mọi tổ hợp** giá trị của n điều kiện con: $2^3 = 8$ test cho evaluate().

#	onCall	newPayee	watchlistHit	$A \wedge B \wedge C$	Quyết định
1	T	T	T	T	PAUSE
2	T	T	F	F	ALLOW
3	T	F	T	F	WARN
4	T	F	F	F	ALLOW
5	F	T	T	F	WARN
6	F	T	F	F	ALLOW
7	F	F	T	F	WARN
8	F	F	F	F	ALLOW

- Đầy đủ nhất nhưng **bùng nổ tổ hợp**: $n = 10$ điều kiện $\Rightarrow 1024$ test.
Cần tiêu chí “vừa đủ mạnh”: MC/DC.

MC/DC — Modified Condition / Decision Coverage

Định nghĩa [2, 4]

Bộ test đạt MC/DC khi:

- 1 Mỗi quyết định nhận cả T và F;
- 2 Mỗi điều kiện con nhận cả T và F;
- 3 Mỗi điều kiện con được chứng minh là **ảnh hưởng độc lập** đến kết quả quyết định: tồn tại **cặp test** chỉ khác nhau ở điều kiện đó mà kết quả quyết định đổi.

Chi phí

Với n điều kiện con chỉ cần tối thiểu $n + 1$ test (thay vì 2^n): với $n = 3$ là **4 test** thay vì 8.

Vì sao ngành hàng không (aviation) chọn MC/DC?

- Chuẩn **DO-178C** [7] (phần mềm điều khiển bay) **bắt buộc** MC/DC cho phần mềm mức A (Level A) — lỗi có thể gây tai nạn thảm khốc.
- Lý do chọn MC/DC:
 - Mạnh hơn hẳn decision/condition coverage: phát hiện lỗi “dây điện đấu nhầm” trong biểu thức logic (&& viết nhầm thành | |, thiếu một điều kiện).
 - Chi phí tuyến tính $n + 1$, khả thi với biểu thức lớn.
- Ứng dụng khác: ô tô (ISO 26262), y tế, đường sắt.

Liên hệ DigiShield

`evaluate()` quyết định **chặn hay cho phép một giao dịch chuyển tiền**
— sai nhánh nghĩa là hoặc chặn nhầm khách hàng, hoặc bỏ lọt giao dịch lừa đảo. Logic “rẻ mà quan trọng” như vậy rất đáng đầu tư MC/DC.

MC/DC cho onCall && newPayee && watchlistHit

Tìm cặp dòng (trong bảng chân trị 8 dòng) chứng minh ảnh hưởng độc lập của từng điều kiện — hai dòng chỉ khác nhau đúng một cột và kết quả D đổi:

Điều kiện	Cặp dòng	Giá trị	D đổi
onCall	#1 vs #5	(T,T,T) vs (F,T,T)	$T \rightarrow F$
newPayee	#1 vs #3	(T,T,T) vs (T,F,T)	$T \rightarrow F$
watchlistHit	#1 vs #2	(T,T,T) vs (T,T,F)	$T \rightarrow F$

Bộ MC/DC tối thiểu

Hợp của các cặp: $\{\#1, \#2, \#3, \#5\}$ — đúng $n + 1 = 4$ test. Dòng #1 được “tái sử dụng” trong cả 3 cặp.

Bộ 4 test MC/DC thành test case DigiShield

#	onCall	newPayee	watchlist	Kỳ vọng	Chứng minh cho
1	true	true	có hit	PAUSE	(cả ba)
2	true	true	không hit	ALLOW	watchlistHit
3	true	false	có hit	WARN	newPayee
5	false	true	có hit	WARN	onCall

```
// Test #5: onCall=false -> không PAUSE, chỉ WARN
var req = new EvaluateRequest(userId,
    new BigDecimal("5000000"), "0123456789",
    false, true); // onCall=false, newPayee=true
// stub watchRepository tra về hit
var decision = service.evaluate(req);
assertThat(decision.decision()).isEqualTo("WARN");
```

Nhận xét: MC/DC “ăn khớp” với nghiệp vụ

- 4 test MC/DC vô tình phủ cả **ba** quyết định nghiệp vụ: PAUSE (#1), ALLOW (#2), WARN (#3, #5).
- Mỗi test trả lời một câu hỏi nghiệp vụ cụ thể:
 - #2: “tài khoản sạch thì dù đang nghe điện thoại vẫn cho chuyển”;
 - #3: “người nhận quen thuộc thì chỉ cảnh báo, không chặn”;
 - #5: “không nghe điện thoại thì chỉ cảnh báo, không chặn”.
- Test class thật `InterceptionServiceImplTest` hiện có 5 test — đối chiếu xem đã đạt MC/DC chưa là bài thực hành.

Kinh nghiệm

Khi biểu thức boolean phức khó nghĩ test, hãy lập bảng chân trị rồi chọn bộ MC/DC — vừa ít test, vừa không bỏ sót vai trò của từng điều kiện.

So sánh các tiêu chí độ phủ điều kiện (1/2)

Áp dụng cho `evaluate()` – chuỗi `if / else if / else` chứa quyết định 3 điều kiện con:

Tiêu chí	Số test tối thiểu	Phát hiện
Statement coverage	3*	dòng chưa chạy
Decision (branch) coverage	3*	nhánh chưa chạy
Condition coverage	2 [†]	điều kiện con chưa lật
Condition/Decision	2 [†]	cả hai mức trên
MC/DC	4 [†]	vai trò từng điều kiện
Multiple condition	8 [†]	mọi tổ hợp

* Tính trên **toàn chuỗi** `if / else if / else`: 3 lệnh gán / 3 lối ra `PAUSE / WARN / ALLOW` loại trừ nhau.

† Tính **riêng trên quyết định phức** `onCall && newPayee && watchlistHit`; nếu tính trên toàn chuỗi thì `Condition/Decision` cũng cần ≥ 3 test (vì bao hàm `decision coverage`).

So sánh các tiêu chí độ phủ điều kiện (2/2)

Thứ tự sức mạnh

Multiple condition $\supset$ MC/DC $\supset$ Condition/Decision $\supset$ Decision $\supset$ Statement.

Độ phức tạp cyclomatic $V(G)$ — McCabe, 1976

Định nghĩa [5]

Số **đường đi độc lập tuyến tính** qua CFG của một hàm — thước đo mức độ “rối” của luồng điều khiển.

Cách 1

$$V(G) = E - N + 2$$

E cạnh, N node của CFG

Cách 2

$$V(G) = P + 1$$

P = số node quyết định
(if, while, for, case...)

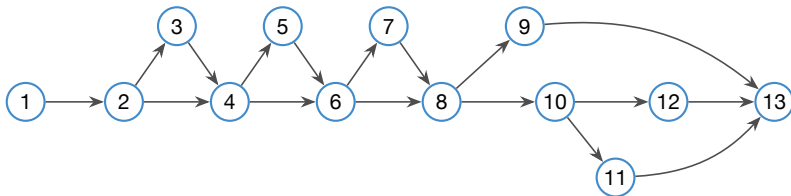
Cách 3

$$V(G) = R$$

R = số miền phẳng khi vẽ CFG (kể cả miền ngoài)

- Ba cách luôn cho cùng kết quả trên CFG hợp lệ (một lối vào, một lối ra).

CFG của evaluate() – đếm E, N



1: khởi tạo signals 2: onCall? 3: add ON_CALL 4:
newPayee? 5: add NEW_PAYEE 6: tra watchlist, hit?
7: add WATCHLIST_HIT 8: cả ba đúng? 9: PAUSE 10:
hit? 11: WARN 12: ALLOW 13: lưu event, return

- $N = 13, E = 17$
 $V(G) = 17 - 13 + 2 = \mathbf{6}$
- Node quyết định: 2, 4, 6, 8, 10
 $V(G) = 5 + 1 = \mathbf{6}$
- Miền phẳng: 5 miền kín + 1 miền ngoài = $\mathbf{6}$

Tính $V(G)$ cho `ageLabel()` — module reporting

```
private String ageLabel(Instant reportedAt,  
                        Instant now) {  
    if (reportedAt == null) {           // Q1  
        return null;  
    }  
    Duration d = Duration  
        .between(reportedAt, now);  
    long mins = Math.max(0, d.toMinutes());  
    if (mins < 60) {                   // Q2  
        return mins + "p";  
    }  
    long hours = d.toHours();  
    if (hours < 24) {                  // Q3  
        return hours + "h";  
    }  
    return d.toDays() + "d";  
}
```

Đếm node quyết định

$P = 3$ (Q1, Q2, Q3)

$V(G) = 3 + 1 = 4$

- 4 đường độc lập = 4 test: `null`, "Np", "Nh", "Nd".
- `Math.max` là lời gọi hàm, không phải nhánh ở mức mã nguồn (JaCoCo cũng không đếm).

Tính $V(G)$ cho computeScore() – module analytics

```
private int computeScore(UUID tenantId,
                          UUID userId) {
    Instant since = Instant.now()
        .minus(SCORING_WINDOW); // 90 ngày
    int signalWeight = riskSignalRepository
        .findByTenantIdAndUserIdAndOccurredAtAfter(
            tenantId, userId, since)
        .stream()
        .mapToInt(RiskSignal::getWeight)
        .sum();
    return Math.max(MIN_RISK, // 0
                    Math.min(MAX_RISK, // 100
                              BASE_RISK + signalWeight));
}
```

Đếm node quyết định

Không có if / vòng lặp

$$V(G) = 0 + 1 = 1$$

Repo tách phần tính điểm sang class

RiskScoring; listing gộp để tiện phân tích.

Nghịch lý

$V(G) = 1$: một test đủ phủ 100% đường đi. Nhưng phép **clamp** 0..100 cần ít nhất 3 test giá trị (âm sâu / giữa / vượt 100)! Độ phủ không thay được phân tích giá trị biên.

Ngưỡng $V(G)$ và số test tối thiểu (1/2)

$V(G)$	Đánh giá (McCabe)	Rủi ro
1–10	Code đơn giản, dễ test	Thấp
11–20	Phức tạp vừa	Trung bình
21–50	Phức tạp, khó test	Cao
> 50	Gần như không thể test đầy đủ	Rất cao

Liên hệ số test case (basis path testing)

$V(G)$ = số đường độc lập tuyến tính = **số test tối thiểu** để phủ tập đường cơ sở (đủ suy ra 100% branch coverage).

Ngưỡng $V(G)$ và số test tối thiểu (2/2)

- DigiShield: `evaluate()` $V = 6$, `ageLabel()` $V = 4$, `computeScore()` $V = 1$ — đều ≤ 10 , đạt chuẩn.
- $V > 10$: cân nhắc **tách hàm** trước khi viết thêm test — refactor rõ hơn test một “hàm quái vật”.
- JaCoCo in sẵn cột Cxty (cyclomatic) trong báo cáo.

Vì sao cần kiểm thử luồng dữ liệu?

Lỗ hổng của độ phủ luồng điều khiển

Branch coverage chỉ hỏi “nhánh này **đã chạy** chưa?” — không hỏi “giá trị được gán ở đây có **đến đúng nơi sử dụng** và **đúng nội dung** không?”

Ví dụ lỗi mà branch coverage bỏ lọt (DigiShield)

Trong `evaluate()`, giả sử lập trình viên quên `signals.add("WATCHLIST_HIT")`: cả 3 nhánh PAUSE/WARN/ALLOW vẫn chạy đủ, test nhánh vẫn xanh — nhưng event lưu xuống DB **thiếu tín hiệu**, đội SOC mất dấu vết điều tra.

Ý tưởng Data Flow Testing

Theo dấu **từng biến**: từ nơi được gán (define) đến mọi nơi được dùng (use), và bắt buộc test đi qua các “đường nối” đó.

def, c-use, p-use

def (define)

Biến được **gán giá trị**: khai báo kèm gán, phép gán, tham số vào hàm, đọc từ input.

c-use

computation use — dùng trong **tính toán**, về phải phép gán, đối số lời gọi hàm, biểu thức return.

p-use

predicate use — dùng trong **điều kiện rẽ nhánh** (if, while, toán tử ba ngôi).

Nhận diện nhanh trên `evaluate()`

- `watchlistHit = hit.isPresent()` — def của `watchlistHit`, c-use của `hit`.
- `if (watchlistHit)` — p-use của `watchlistHit`.
- `String.join(",", signals)` — c-use của `signals`.

DU-pair và DU-path

DU-pair (cặp def-use)

Cặp (d, u) : vị trí d định nghĩa biến x và vị trí u sử dụng x , sao cho tồn tại đường đi từ d đến u **không có định nghĩa lại** x ở giữa (đường *definition-clear*).

DU-path

Một đường đi definition-clear **cụ thể** trong CFG từ d đến u . Một DU-pair có thể có nhiều DU-path (đi qua các nhánh khác nhau).

- Nếu giữa d và u có một phép gán khác cho x — def cũ bị **kill**, cặp (d, u) không còn hiệu lực.
- Test một DU-pair = tạo dữ liệu vào sao cho chương trình **đi qua** d rồi tới u , và **assert** giá trị tại u .

Ba tiêu chí độ phủ luồng dữ liệu

Tiêu chí	Yêu cầu	Chi phí
all-defs	mỗi def tới ít nhất một use	thấp
all-uses	mỗi DU-pair được thực thi ít nhất 1 lần	vừa
all-DU-paths	mọi DU-path của mọi DU-pair	cao

Quan hệ bao hàm

$\text{all-DU-paths} \supset \text{all-uses} \supset \text{all-defs}$;
all-uses cũng mạnh hơn branch coverage
(mọi p-use kéo theo cả hai nhánh).

Thực dụng

Trong công nghiệp thường nhắm **all-uses** cho biến “quan trọng” (tiền, quyết định, trạng thái).

Ví dụ 1: biến signals trong evaluate()

```
List<String> signals = new ArrayList<>(); // d1: DEF
if (request.onCall())
    signals.add("ON_CALL"); // d2: c-USE + DEF lại
if (request.newPayee())
    signals.add("NEW_PAYEE"); // d3: c-USE + DEF lại
if (watchlistHit)
    signals.add("WATCHLIST_HIT"); // d4: c-USE + DEF lại
// ... quyết định PAUSE/WARN/ALLOW ...
new InterventionEvent(...,
    String.join(",", signals), ...); // u1: c-USE
return new InterventionDecision(
    decision.name(), signals, message); // u2: c-USE
```

- signals **không có p-use** — không bao giờ xuất hiện trong điều kiện.
- Mỗi add () vừa dùng (c-use) vừa làm thay đổi trạng thái (def lại) — quy ước đơn giản hóa cho biến tập hợp.

DU-pairs của signals → test case

DU-pair	Đường def-clear	Test case	Assert tại use
$(d1, u1/u2)$	không add nào chạy	ALLOW “sạch”	signals rỗng
$(d2, u1/u2)$	chỉ onCall	ALLOW + ON_CALL	chỉ chứa ON_CALL
$(d3, u1/u2)$	chỉ newPayee	ALLOW + NEW_PAYEE	chỉ chứa NEW_PAYEE
$(d4, u1/u2)$	chỉ hit	WARN	chứa WATCHLIST_HIT

- $u1$ = chuỗi lưu vào InterventionEvent (DB); $u2$ = danh sách trả về cho client. **Phải assert cả hai** — chúng có thể lệch nhau nếu code sửa một nơi quên nơi kia.
- Test thật dùng ArgumentCaptor<InterventionEvent> để bắt $u1$ — xem slide “Cài đặt data flow test bằng ArgumentCaptor” ngay sau.

Ví dụ 2: safePage, safeSize trong listInterventions()

```
public List<InterventionEventView>
    listInterventions(int page, int size) {
    UUID tenantId = TenantContext.requireUuid();
    int safePage = Math.max(page, 1);           // DEF safePage
    int safeSize = Math.min(
        Math.max(size, 1), 100);               // DEF safeSize
    Pageable pageable = PageRequest.of(
        safePage - 1, safeSize);               // c-USE cả hai biến
    return eventRepository
        .findByTenantIdOrderByTsDesc(
            tenantId, pageable)                 // c-USE pageable
        .stream().map(...).toList();
}
```

- Mỗi biến chỉ có **một** DU-pair (def → PageRequest), nhưng **giá trị** def là hàm clamp phi tuyến của input.
- Data flow nói “test cặp def–use”; BVA (buổi 8) nói **giá trị nào**: $\text{size} \in \{0, 1, 2, 99, 100, 101\}$.

Cài đặt data flow test bằng ArgumentCaptor

Từ InterceptionServiceImplTest (test thật, rút gọn)

```
@Captor
ArgumentCaptor<InterventionEvent> eventCaptor;

@Test
void warnKhiChiCoWatchlistHit() {
    // ... stub watchRepository co hit ...
    service.evaluate(new EvaluateRequest(
        userId, amount, "0123456789",
        false, false)); // onCall=F, newPayee=F

    verify(eventRepository).save(
        eventCaptor.capture());
    // Assert tại điểm USE: dữ liệu lưu DB đúng
    assertThat(eventCaptor.getValue().getSignals())
        .isEqualTo("WATCHLIST_HIT");
}
```

Captor “chụp” giá trị tại điểm use — đúng tinh thần data flow: kiểm tra **dữ liệu đến nơi**, không chỉ “hàm được gọi”.

Bài tập trên lớp: phân tích computeScore()

Đề bài (làm theo cặp, 10 phút)

Đánh số dòng, lập bảng def / c-use / p-use cho **mọi biến cục bộ**, liệt kê các DU-pair và đề xuất test case cho từng cặp.

```
private int computeScore(UUID tenantId,
                        UUID userId) {
    Instant since = Instant.now()
        .minus(SCORING_WINDOW);           // L1
    int signalWeight = riskSignalRepository
        .findByTenantIdAndUserIdAndOccurredAtAfter(
            tenantId, userId, since)       // L2
        .stream()
        .mapToInt(RiskSignal::getWeight)
        .sum();                             // L3
    return Math.max(MIN_RISK,
        Math.min(MAX_RISK,
            BASE_RISK + signalWeight));    // L4
}
```

Lời giải: bảng def / use theo dòng

Biến	def	c-use	p-use	DU-pair
since	L1	L2 (đối số truy vấn)	—	(L1, L2)
signalWeight	L3	L4 (biểu thức return)	—	(L3, L4)
tenantId	tham số	L2	—	(vào, L2)
userId	tham số	L2	—	(vào, L2)

- Không biến nào có p-use — khớp với $V(G) = 1$.
- BASE_RISK, MIN_RISK, MAX_RISK là hằng số (static final) — def một lần lúc nạp class.
- DU-pair (L1, L2) kiểm chứng **cửa sổ 90 ngày**: tín hiệu cũ hơn since không được tính.

Lời giải: từ DU-pairs đến test case

DU-pair	Test case	Kỳ vọng
(L1, L2)	stub repo, verify đối số since	$\approx$ now – 90 ngày
(L3, L4) tổng dương vừa	signals nặng +20	score = $5 + 20 = 25$
(L3, L4) tổng âm sâu	signals –30	clamp về 0
(L3, L4) tổng rất lớn	signals +200	clamp về 100
(L3, L4) không signal	repo trả list rỗng	score = BASE_RISK = 5

Nhận xét

Một DU-pair (L3, L4) sinh ra **4 test** vì giá trị tại use có 3 miền (dưới 0 / trong / trên 100) + trường hợp mặc định. Data flow chỉ ra **chỗ** phải test; EP/BVA chỉ ra **giá trị** — hai kỹ thuật bổ sung nhau.

Ba loại anomaly luồng dữ liệu

Anomaly = chuỗi def/use **bất thường**, thường là triệu chứng của bug hoặc code thừa:

Loại	Mẫu	Triệu chứng của
define–define (dd)	gán rồi gán đè, chưa hề dùng	quên dùng giá trị đầu; sai thứ tự lệnh
define–kill (dk)	gán rồi ra khỏi scope, không dùng	biến thừa; logic bị xóa dờ
use-before-define (ud)	dùng khi chưa chắc đã gán	thiếu nhánh khởi tạo; NPE

- Anomaly phát hiện được bằng **phân tích tĩnh** — không cần chạy chương trình.
- Java: compiler chặn “dùng biến local chưa gán”, nhưng **không** chặn field/đối tượng null — vẫn cần soi.

Ví dụ anomaly và công cụ phát hiện

```
// 1. dd: gan de len khi chua dung -> mat gia tri
int score = computeScore(tenantId, userId);
score = 0;                                // bug? hay co y?

// 2. dk: dinh nghia roi khong ai dung
int unused = scores.size();    // code chet

// 3. ud: dung khi co the chua gan
Integer risk = null;
if (highRisk) { risk = 90; }
return risk + 5;    // NPE khi highRisk = false
```

- IDE (IntelliJ) cảnh báo ngay: *unused assignment, variable is never used, unboxing may produce NPE*.
- CI của DigiShield chạy **Checkstyle** (buildSrc convention); có thể bổ sung SpotBugs / ErrorProne để bắt anomaly sâu hơn.
- Bài học: đọc warning của IDE = data flow analysis miễn phí.

Cách hoạt động [6]

JaCoCo gắn một **Java agent** vào JVM khi chạy test; agent *instrument* bytecode để đếm mỗi instruction / nhánh đã thực thi, rồi xuất báo cáo HTML + XML.

- Đo ở mức **bytecode** $\Rightarrow$ thấy được từng điều kiện con của `&&` / `||` (liên hệ condition coverage).
- Tích hợp sẵn với Gradle qua plugin `jacoco` — DigiShield đã bật cho **mọi module** qua convention plugin trong `buildSrc`.
- File cấu hình thật:
`buildSrc/src/main/kotlin/
digishield.spring-module-conventions.gradle.kts`

Cấu hình thật (1): plugin và report

Trích digishield.spring-module-conventions.gradle.kts

```
plugins {  
    `java-library`  
    checkstyle  
    jacoco // bat coverage moi module  
    id("io.spring.dependency-management")  
}  
  
tasks.jacocoTestReport {  
    dependsOn(tasks.test)  
    reports {  
        xml.required.set(true) // cho CI/SonarQube  
        html.required.set(true) // cho nguoi doc  
    }  
}  
  
tasks.test {  
    finalizedBy(tasks.jacocoTestReport)  
    // chay test xong tu dong sinh report  
}
```

Cấu hình thật (2): ngưỡng coverage gắn vào check

```
tasks.jacocoTestCoverageVerification {
    violationRules {
        rule {
            limit {
                counter = "LINE"
                value = "COVEREDRATIO"
                // TODO raise threshold as coverage
                // grows (target 0.50).
                minimum = "0.10".toBigDecimal()
            }
        }
    }
}

tasks.named("check") {
    dependsOn(tasks.jacocoTestCoverageVerification)
}
```

• `./gradlew check` **fail** nếu line coverage của module xuống dưới 10% — chạy trong CI cho mọi pull request.

Thảo luận: vì sao ngưỡng chỉ 0.10?

Chiến lược “ratchet” (bánh cóc)

- 1 Đặt ngưỡng **thấp hơn hiện trạng** một chút $\Rightarrow$ build luôn xanh.
- 2 Ngưỡng chặn **tụt lùi**: code mới không test sẽ kéo tỉ lệ xuống và fail.
- 3 Coverage tăng dần $\Rightarrow$ **nâng ngưỡng theo** (TODO trong code: 0.50).

Vì sao không đặt 0.80 ngay?

- Dự án đang phát triển, 375 test / 9 module — ép 80% ngay là **chặn mọi PR**.
- Đội sẽ viết test đối phó (không assertion) chỉ để qua cổng — phản tác dụng.

Câu hỏi thảo luận

Nên nâng ngưỡng theo lịch (mỗi sprint +5%) hay theo hiện trạng (ngưỡng = coverage hiện tại – 2%)? Ưu nhược của từng cách?

Cấu hình thật (3): loại trừ khỏi coverage

```
val jacocoExclusions = listOf(  
    "**/*Application*", // main class  
    "**/config/**",     // Spring @Configuration  
    "**/*Config*",  
    "**/package-info*",  
)
```

Loại trừ vì:

- Config/Application là code khai báo, unit test không có gì để assert.
- Giữ lại sẽ “pha loãng” tỉ lệ, che mất class nghiệp vụ thiếu test.

Nguyên tắc: chỉ loại trừ code **không thể** unit test có ý nghĩa; không loại trừ để “làm đẹp số”.

Cố tình giữ lại:

- application/** (các *ServiceImpl) — nơi chứa nghiệp vụ, chính là chỗ cần đo.
- Comment trong file thật ghi rõ điều này.

Chạy và tìm báo cáo

```
# Test 1 module -> report tu sinh (finalizedBy)
./gradlew :modules:interception:test

# Report HTML của module:
# modules/interception/build/reports/jacoco/
#   test/html/index.html

# Report tổng hợp toàn hệ thống:
./gradlew testCodeCoverageReport
# build/reports/jacoco/
#   testCodeCoverageReport/html/index.html

# Cong chat luong (fail neu LINE < 10%):
./gradlew check
```

- Mỗi module một report riêng + một report gộp cho cả hệ thống — BTL dùng report **module nhóm mình**.

Đọc báo cáo JaCoCo

Các cột trong report:

- **Instructions:** bytecode đã chạy / tổng
- **Branches:** nhánh (if, &&, switch)
- **Cxty:** độ phức tạp cyclomatic!
- **Lines:** dòng nguồn — cột mà ngưỡng 0.10 đang đo
- **Methods / Classes:** đã được gọi / nạp

Màu trong source view:

- **Xanh lá:** phủ hoàn toàn
- **Vàng:** nhánh phủ **một phần** — ví dụ && 3 điều kiện mới lật được 4/6 nhánh bytecode
- **Đỏ:** chưa chạy lần nào

Kim chỉ nam đọc report

Vào package `application` → sắp xếp theo **Missed Branches** giảm dần
→ dòng **vàng** là mỏ vàng: thường là điều kiện phức thiếu test MC/DC.

Kịch bản demo: “đục lỗ” rồi vá nhánh WARN

InterceptionServiceImplTest có sẵn 5 test phủ đủ 3 nhánh. Ta **tạm tắt** test nhánh WARN để thấy report đổi màu:

Bước 1–3: tạo lỗ hổng coverage

```
# B1. Chạy test, mo report -> evaluate() xanh hết
./gradlew :modules:interception:test
open modules/interception/build/reports/\
jacoco/test/html/index.html
```

```
# B2. Trong InterceptionServiceImplTest.java:
#     them @Disabled vào test của nhánh WARN
```

```
# B3. Chạy lại
./gradlew :modules:interception:test --rerun-tasks
```

Quan sát sau B3: dòng else if (watchlistHit) chuyển **vàng/đỏ**; cột Branches của evaluate() giảm; message WARN thành dòng **đỏ**.

Kịch bản demo: vá lại và đối chiếu

Bước 4–5: vá và kiểm chứng

- B4.** Bỏ `@Disabled`, chạy lại `./gradlew :modules:interception:test --rerun-tasks` → nhánh WARN xanh trở lại.
- B5.** Mở class `InterceptionServiceImpl` trong report: đối chiếu cột **Cxty** của `evaluate()` với $V(G) = 6$ ta tính tay (JaCoCo tính trên bytecode nên có thể lệch nhẹ — giải thích vì sao).

Điều cần rút ra

- Report là **la bàn**: nhìn màu là biết viết test gì tiếp.
- Xóa 1 test $\Rightarrow$ mất hẳn 1 nhánh nghiệp vụ (WARN): mỗi test đang “gác” một hành vi thật.
- `--rerun-tasks` ép Gradle chạy lại dù không đổi code (Gradle cache kết quả test).

100% coverage $\neq$ hết lỗi

Nguyên tắc ISTQB (buổi 1) [4]

#1 — Kiểm thử chỉ chứng minh **sự hiện diện** của lỗi, không chứng minh sự vắng mặt. #2 — Kiểm thử vét cạn là **bất khả thi**. Coverage 100% vẫn chỉ là một mẫu rất nhỏ của không gian input.

Ba kiểu lỗi “miễn nhiễm” với coverage — ví dụ DigiShield:

- **Lỗi giá trị:** `computeScore()` coverage 100% với 1 test, nhưng nếu ai đổi `MAX_RISK` thành 1000 thì clamp sai mà test dòng lệnh vẫn xanh.
- **Lỗi thiếu code:** quên `signals.add("WATCHLIST_HIT")` — code không tồn tại thì không có dòng nào để “chưa phủ”.
- **Test không assert:** gọi `evaluate()` mà không `assertThat` — coverage tăng, giá trị bằng 0.

Coverage là bản đồ, không phải đích đến

Dùng đúng

- Tìm **chỗ chưa test**: class đỏ, nhánh vàng.
- Chặn **tụt lùi** qua cổng check.
- So sánh **xu hướng** giữa các sprint.

Dùng sai

- KPI “phải đạt 90%” → test rác, assert hời hợt.
- Khoe con số thay vì chất lượng assertion.

Định luật Goodhart

“Khi một thước đo trở thành mục tiêu, nó không còn là thước đo tốt nữa.”
Coverage đo *code đã chạy*, không đo *hành vi đã được kiểm chứng*.
Mutation testing (PIT) là bước kiểm tra chất lượng test — giới thiệu ở buổi 12.

Thực hành trên lớp: săn nhánh thiếu trong analytics

Nhiệm vụ (30 phút, làm theo cặp)

- 1 Chạy test + sinh report cho module analytics:

```
./gradlew :modules:analytics:test  
open modules/analytics/build/reports/\  
jacoco/test/html/index.html
```

- 2 Vào package `com.digishield.analytics.application`, mở `AnalyticsServiceImpl`, sắp xếp theo **Missed Branches**.
- 3 Ghi lại: method nào có branch coverage **thấp nhất**? Nhánh nào đang vàng/đỏ?
- 4 Viết **2 test mới** trong `AnalyticsServiceImplTest` để phủ các nhánh đó, chạy lại và chụp màn hình trước / sau.

Gợi ý: các nhánh hay bị bỏ sót trong AnalyticsServiceImpl

- `benchmark(scope)`: nhánh `scores.isEmpty()` trả về 0 — đã có test chưa?
- `riskFor(scope, scopeId)`: hai nhánh `scopeId != null / null` và nhánh `latest == null` — tổ hợp nào còn thiếu?
- `orgPhishPronePct(...)`: lấy `phishPronePct` gần nhất từ lịch sử rollup, nhánh `orElse(0.0)` khi chưa có — nhánh `private`, phủ gián tiếp qua `dashboard()` hoặc `benchmarkRates()`.

Tiêu chí nghiệm thu 2 test mới

(1) Có assertion **giá trị** (không chỉ verify gọi hàm); (2) tên test nói rõ nhánh nào (vd: `benchmarkTraVe0KhiChuaCoDiemNao`); (3) Missed Branches của method giảm thật trong report.

Nhắc lại buổi 3: dựng test theo mẫu `MockitoExtension` + `@Mock repository` + `TenantContext` như các test có sẵn.

Bài nộp (theo nhóm BTL — hạn: trước buổi 5)

Báo cáo độ phủ module nhóm phụ trách (PDF, 3–5 trang)

- ❶ **Hiện trạng:** bảng Line / Branch coverage theo class (chụp từ report JaCoCo của module nhóm), nêu 3 class thấp nhất.
- ❷ **Phân tích trắng:** chọn 2 method phức tạp nhất — tính $V(G)$ (ghi rõ cách tính), lập bảng DU-pairs cho 1 biến quan trọng của mỗi method.
- ❸ **Kế hoạch nâng:** danh sách test case sẽ viết (tên + nhánh/DU-pair nhắm tới) để đưa Line coverage của module lên $\geq 50\%$; ước lượng số test.
- ❹ **Đã làm:** commit 2 test của bài thực hành (analytics hoặc module nhóm) kèm ảnh coverage trước / sau.
 - Nộp: push nhánh bt1/nhom-XX + PDF lên LMS.
 - Báo cáo này là **mục 3 của BTL** (thiết kế test hộp trắng) — sẽ chấm trong 30% điểm BTL.

Tổng kết buổi 4 — khép lại Chương 3

Hôm nay

- Condition / MC/DC: 4 test “soi” đủ 3 điều kiện của `evaluate()`
- $V(G)$: 3 cách tính; `evaluate` = 6, `ageLabel` = 4, `computeScore` = 1
- Data flow: `def` / `c-use` / `p-use`, `DU-pair`, `all-defs` / `all-uses` / `all-DU-paths`, `anomaly`
- JaCoCo: `convention` plugin, ngưỡng 0.10, đọc màu và cột report

Buổi 5 (Chương 4)

- JUnit 5 chuyên sâu: `lifecycle`, `assertions`, `AssertJ`, `parameterized test`
- Biến các thiết kế test hôm nay thành code chạy được

Mang theo

Laptop đã chạy được `./gradlew test` và report JaCoCo của module nhóm.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt. *Introduction to Software Testing*. Cambridge University Press, 2008 — Ch. 7 (Graph Coverage), Ch. 8 (Logic Coverage / MC-DC).
- [3] Glenford J. Myers. *The Art of Software Testing*, 2nd ed., Wiley, 2004 — Ch. 4 (Test-Case Design).
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] T. J. McCabe. “A Complexity Measure”. *IEEE Trans. on Software Engineering*, SE-2(4), 1976.
- [6] JaCoCo Documentation. <https://www.jacoco.org/jacoco/trunk/doc/> (counters, Gradle plugin).
- [7] RTCA DO-178C. *Software Considerations in Airborne Systems and Equipment Certification* (bối cảnh MC/DC).